Proyecto 2 – 2026 – Data Memory Hierarchy

Mandzyak Melnyk, Mijayl

Manzano Miranda, Héctor

Lunes, 11 de mayo de 2026

Breve Resumen

Esta memoria presenta el segundo proyecto realizado en la asignatura de AOC2 consistente en completar el diagrama de estados en vhdl de la Unidad de Control de la memoria cache, esta memoria cache posee un tamaño de 128 bytes (32 palabras), 2 vías de set asociativas, una política de reemplazo de tipo FIFO(First-In,First-Out), con una política de escritura "CopyBack" o también llamado "Write-Back" y una política de escritura en caso de fallo en caché de tipo "Write-Around"(NANF).

En este proyecto hemos completado la Unidad de Control de la memoria caché con 6 estados, reutilizando algunos para diferentes casos. Además, hemos realizado 8 pruebas para comprobar el correcto funcionamiento del proyecto y realizamos 2 pruebas de integración para observar 1 prueba con SlowDown otra con SpeedUp respecto a no tener caché ni scratch.

Diagrama de estados de la unidad de control

Hemos decidido liberar el bus al acabar el Copyback y previamente a escribir el bloque en caché, debido a que esto provoca una mayor eficiencia en memoria permitiendo al arbitraje ceder el bus al IO_MASTER.

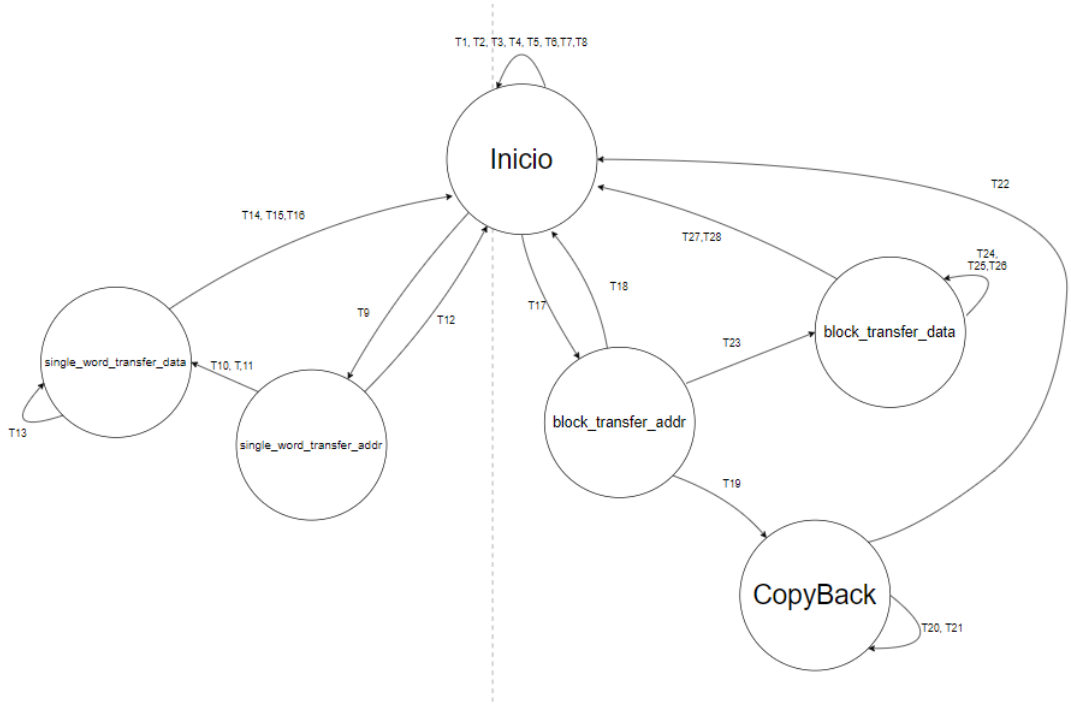


Tabla de Transiciones de la Unida de Control		
Estado Actual	**Condición de Entrada (Señales)**	**Señales Activadas**
T1	RE = ‘0’ AND WE =’0’	ready='1'
T2	(RE='1' OR WE='1') AND unaligned='1'	ready='1',
T3	RE='1' AND internal_addr='1'	ready='1', mux_output="10"
T4	WE='1' AND internal_addr='1'	ready='1',
T5	RE='1' AND hit='1'	ready='1', inc_r='1', mux_output="00"
T6	WE='1' AND hit='1' AND hit0=’1’	ready='1', inc_w='1', MC_WE0 ='1', Update_dirty='1'
T7	WE='1' AND hit='1' AND hit1=’1’	ready='1', inc_w='1', MC_WE1 ='1', Update_dirty='1'
T8	(RE= '1') OR (WE='1') AND hit='0'	Bus_req =’1’, Bus_grant=’0’
T9	(addr_non_cacheable = '1' OR WE = '1') AND Bus_grant= ‘1’ AND Hit=’0’	Bus_req = ‘1’, One_Word=’1’
T10	RE=’1’ AND Bus_DevSel=’1’	Frame=’1’, MC_send_addr_ctrl=’1’, MC_bus_Read=’1’,

T11	WE=’1’ AND Bus_DevSel=’1’	Frame=’1’, MC_send_addr_ctrl=’1’, MC_bus_Write=’1’,
T12	RE=’1’ OR WE=’1’ AND Bus_DevSel=’0’	Frame=’1’, MC_send_addr_ctrl=’1’, load_addr_error=’1’, ready=’1’
T13	Bus_TDRY=’0’	Frame=’1’
T14	Bus_TDRY=’1’ AND RE=’1’ AND Addr_non_cacheable=’1’	Frame=’1’, Last_word=’1’, Mux_output=’01’, ready='1'
T15	Bus_TDRY=’1’ AND WE=’1’ AND Addr_non_cacheable=’1’	Frame=’1’, Last_word=’1’, MC_send_data=’1’, ready='1'
T16	Bus_TDRY=’1’ AND WE=’1’ AND Addr_non_cacheable=’0’	Frame=’1’, Last_word=’1’, MC_send_data=’1’, Inc_m=’1’, ready='1'
T17	Bus_grant=’1’ AND RE=’1’ AND Addr_non_cacheable=’0’	Bus_req = ‘1’
T18	Bus_DevSel= ’0’	MC_send_addr_ctrl=’1’, Frame=’1’, Block_addr=’1’, Load_addr_error=’1’, Ready=’1’
T19	Dirty_bit_rpl=’1’ AND Bus_DevSel=’1’	MC_send_addr_ctrl=’1’, Frame=’1’, Block_addr=’1’, MC_bus_Write=’1’, Send_dirty=’1’,
T20	bus_TRDY='0'	Frame = ‘1’, MC_send_data = ‘1’, MC_bus_Write = ‘1’, Send_dirty = ‘1’, mux_origen=’1’

T21	bus_TRDY='1' AND last_word_block='0'	Frame = '1', MC_send_data = '1', MC_bus_Write = '1', Send_dirty = '1', mux_origen='1' Count_enable='1'
T22	bus_TRDY='1' AND last_word_block='1'	Frame='1' MC_send_data = '1', MC_bus_Write = '1', Send_dirty = '1', Mux_origen='1' Last_word = '1', Inc_cb = '1', Block_copied_back = '1', Update_dirty = '1'
T23	Dirty_bit_rpl='0' AND Bus_DevSel='1'	MC_send_addr_ctrl='1', Frame='1', Block_addr='1', MC_bus_Read='1'
T24	Bus_TDRY='0'	Frame='1', Mux_origen='1'
T25	Bus_TDRY='1' AND Via_2_rpl='1'	Frame='1', Mux_origen='1', Count_enable='1', MC_WE1='1'
T26	Bus_TDRY='1' AND Via_2_rpl='0'	Frame='1', Mux_origen='1', Count_enable='1', MC_WE0='1'
T27	Bus_TDRY='1' AND Via_2_rpl='0' AND Last_Word_Block='1'	Frame='1', Mux_origen='1', MC_WE0='1', Last_word='1', MC_tags_WE='1', Inc_m='1'
T28	Bus_TDRY='1' And Via_2_rpl='1' And Last_Word_Block='1'	Frame='1', Mux_origen='1', MC_WE1='1', Last_word='1', MC_tags_WE='1', Inc_m='1'

Desglose de direcciones

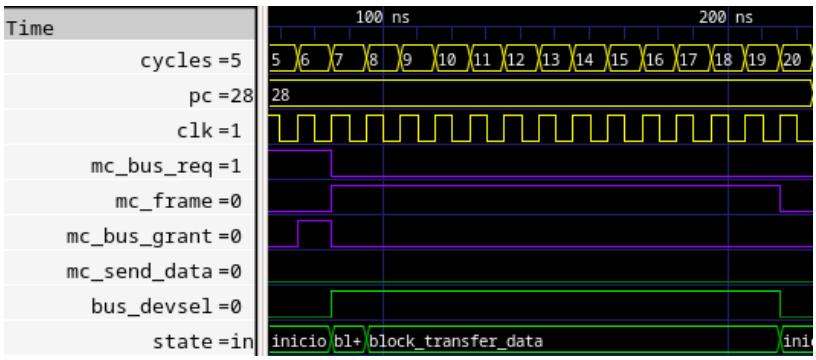
En la guía se nos informa que la MC es una caché asociativa de dos vías de 128 bytes (32 palabras) y que cada vía almacena 4 bloques de 4 palabras. Para distinguir el byte dentro de la palabra (de 32 bits) necesitamos 2 bits. Como cada bloque contiene 4 palabras necesitamos otros 2 bits para distinguirla. Para poder diferenciar el conjunto también necesitamos 2 bits. Por tanto, tenemos 26 bits para el tag (32 - 2 - 2 - 2).

De esta manera, tenemos direcciones de 32 bits que se dividen de la siguiente manera:

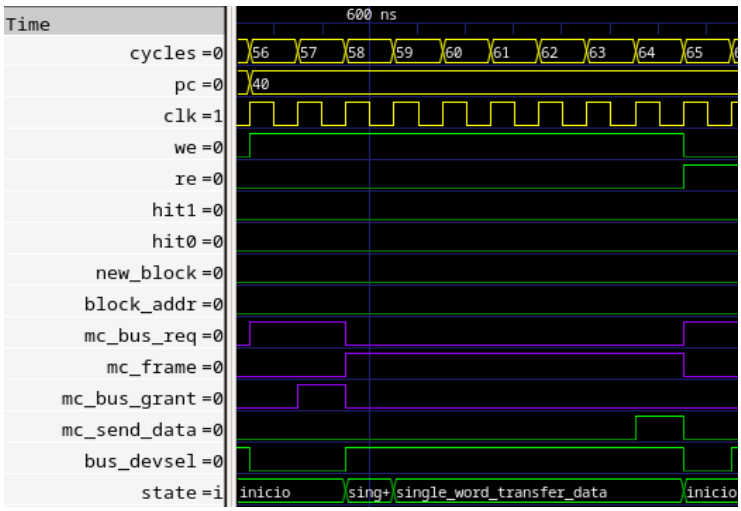
[26bits de tag | 2 bits de set | 2 bits de palabra | 2 bits de byte]

Análisis de la latencia en la transferencia de datos por bus

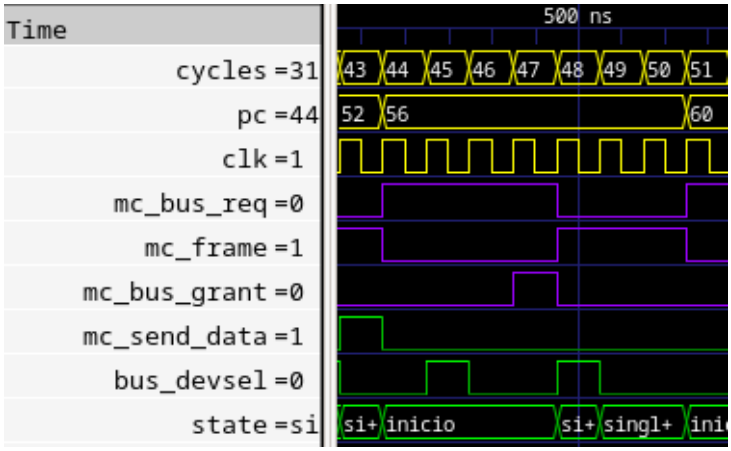
CrB (MD) = 1(addr)+6 (L/word1) + 2 (word2) +2(word3) +2(word4) = 13 ciclos



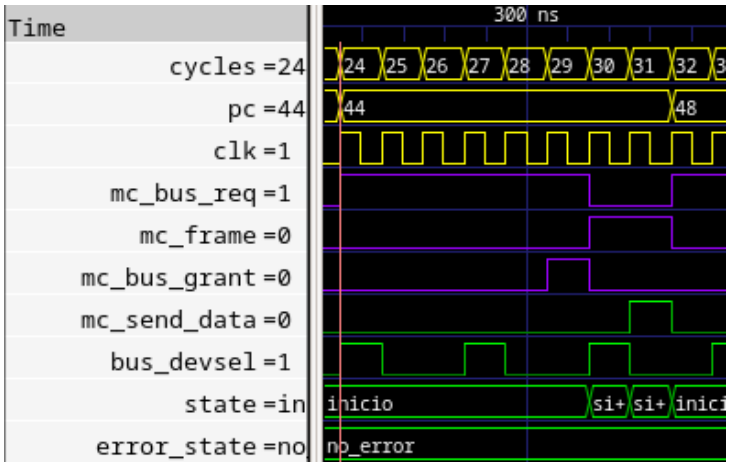
CwW(MD) = 1(addr) + 6(L) = 7 ciclos



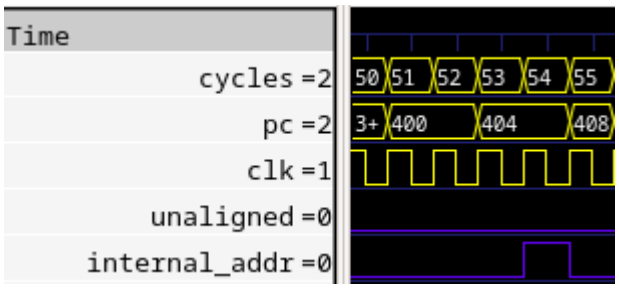
CrW(MDscratch) = 1(addr)+2 = 3 ciclos



CwW (MDscratch) = 1(addr) + 1= 2 ciclos



Crw/CwW (IO_REG) = 1 ciclo



Ciclos promedio por acceso a memoria

$$C_{eff} = 1 + P_{rm} \cdot (CrB + T_{arb}) + P_{(m \wedge dirty)} \cdot (CwW + T_{arb}) + P_{wm} \cdot (CwW + T_{arb}) + P_{rsc} \cdot (CrW + T_{arb}) + P_{wsc} \cdot (CwW + T_{arb}) + P_{rio} \cdot (CrW) + P_{wio} \cdot (CwW) + P_{wh} \cdot (CwW) + P_{rh} \cdot (CrW)$$

Suponemos un caso hipotético real en el que al haber localidad temporal y espacial se producirían grandes cantidades de hits. Además, en la ecuación no sumaremos el lookup debido a que en el apartado anterior tenemos en cuenta el ciclo del addr.

Suponemos que los wh y rh son el 30% de las instrucciones cada una. El rm dirty el 10% y el rm clean otro 10%. Tanto wsc y rsc un 4% cada una. Y rio y wio un 1% cada una.Y un 10% de wm.

$$C_{eff} = 0{,}10 \cdot (13 + 1{,}5) + 0{,}10 \cdot (13 + 1{,}5) + 0{,}10 \cdot (7 + 1{,}5) + 0{,}04 \cdot (3 + 1{,}5) + 0{,}04 \cdot (2 + 1{,}5) + 0{,}01 \cdot (1) + 0{,}01 \cdot (1) + 0{,}3 \cdot (1) + 0{,}3 \cdot (1) = 4{,}69$$

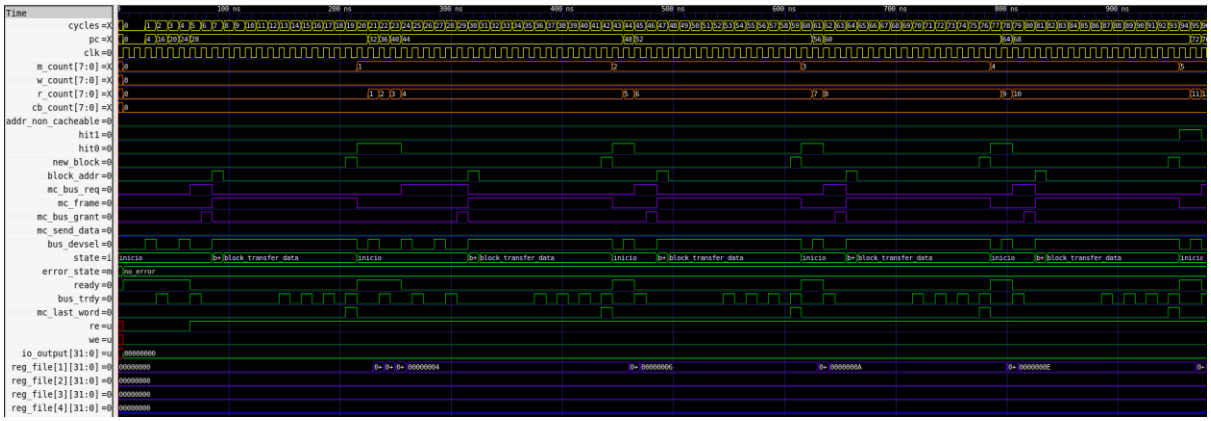
Pruebas Unitarias

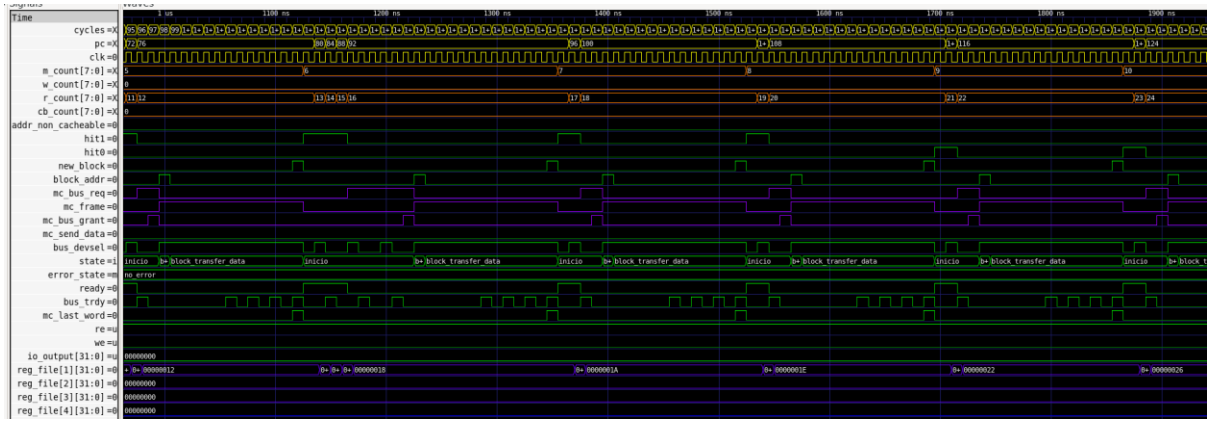
Hemos adjuntado un txt llamado “Pruebas.txt” en el que se incluyen todas las ram de memoria e instrucciones usadas para cada prueba. Todos los tests los hemos ejecutado con las irq desactivadas.

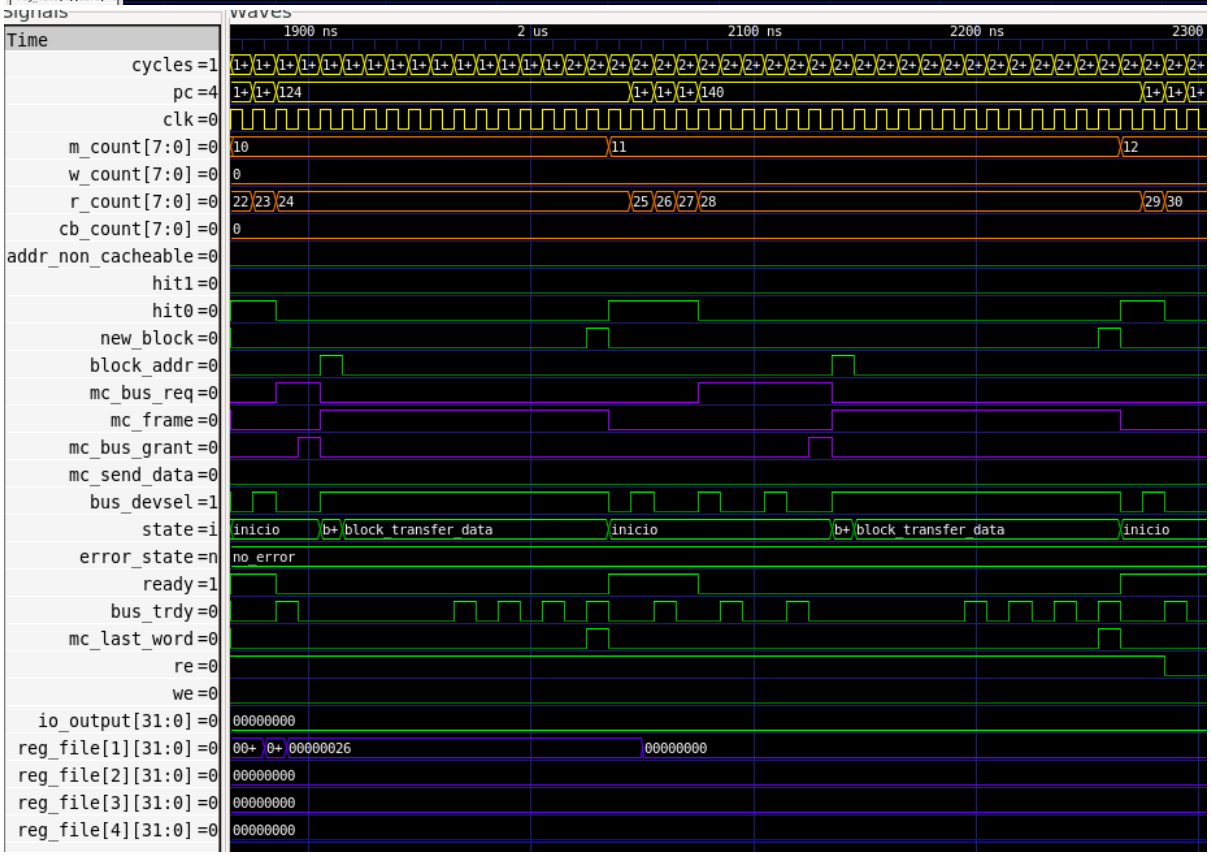
	FSM	Pruebas
falloLecAciertoLecs	Inicio, block_transfer_addr, block_transfer_data	Read Miss Clean, Read Hit
scratch	Inicio, block_transfer_addr, block_transfer_data, single_word_transfer_addr, single_word_transfer_data	Read Miss Clean, Read Hit, Write Scratch, Read Scratch
missDirty	Inicio, block_transfer_addr, block_transfer_data, CopyBack	Read Miss Clean, Write Hit, Read Miss Dirty
writeHit	Inicio, block_transfer_addr, block_transfer_data	Read Miss Clean, Write Hit
writeMiss	Inicio, block_transfer_addr, block_transfer_data, single_word_transfer_addr, single_word_transfer_data	Read Miss Clean, Write Miss
errores	Inicio, block_transfer_addr, block_transfer_data	Read Miss, Read hit, Internal_Addr, Out of range, Unaligned
completo	Inicio, block_transfer_addr, block_transfer_data, single_word_transfer_addr, single_word_transfer_data, CopyBack	Read Miss Clean, Write Hit, Write Miss, Write Scratch, Read Scratch, Read Miss Dirty, Read Hit, Read Hit, Unaligned, Write IO
bucle	Inicio, block_transfer_addr, block_transfer_data, single_word_transfer_addr, single_word_transfer_data	Read Miss, Write Scratch, Read Hit, Read Scratch

Prueba falloLecAciertoLecs(2300ns)

```
-- Vía 0
-- Set 0
LW R1, 0(R0) -- Read Miss Clean (Carga Vía 0). R1 -> X"00000001"
LW R1, 4(R0) -- Read Hit. R1 -> X"00000002"
LW R1, 8(R0) -- Read Hit. R1 -> X"00000003"
LW R1, 12(R0) -- Read Hit. R1 -> X"00000004"

-- Set 1 LW R1, 16(R0)
-- Read Miss Clean (Carga Vía 0). R1 -> X"00000005"
LW R1, 20(R0) -- Read Hit. R1 -> X"00000006"

-- Set 2 LW R1, 32(R0) -- Read Miss Clean (Carga Vía 0). R1 -> X"00000009"
LW R1, 36(R0) -- Read Hit. R1 -> X"0000000A"

-- Set 3 LW R1, 48(R0) -- Read Miss Clean (Carga Vía 0). R1 -> X"0000000D"
LW R1, 52(R0) -- Read Hit. R1 -> X"0000000E"

-- Vía 1 -- Set 0
LW R1, 64(R0) -- Read Miss Clean (Carga Vía 1). R1 -> X"00000011"
LW R1, 68(R0) -- Read Hit. R1 -> X"00000012"

-- Set 1
LW R1, 80(R0) -- Read Miss Clean (Carga Vía 1). R1 -> X"00000015"
LW R1, 84(R0) -- Read Hit. R1 -> X"00000016"
LW R1, 88(R0) -- Read Hit. R1 -> X"00000017"
LW R1, 92(R0) -- Read Hit. R1 -> X"00000018"

-- Set 2 LW R1, 96(R0) -- Read Miss Clean (Carga Vía 1). R1 -> X"00000019"
LW R1, 100(R0) -- Read Hit. R1 -> X"0000001A"

-- Set 3 LW R1, 112(R0) -- Read Hit Miss Clean (Carga Vía 1). R1 -> X"0000001D"
LW R1, 116(R0) -- Read Hit. R1 -> X"0000001E"

-- Reemplazo de la vía 0 en todos los conjuntos -- Set 0
LW R1, 128(R0) -- Read Hit Miss Clean (Reemplazo). R1 -> X"00000021"
LW R1, 132(R0) -- Read Hit Hit. R1 -> X"00000022"

-- Set 1 LW R1, 144(R0) -- Read Miss Clean (Reemplazo). R1 -> X"00000025"
LW R1, 148(R0) -- Read Hit. R1 -> X"00000026"

-- Set 2
LW R1, 160(R0) -- Read Miss Clean (Reemplazo). R1 -> X"00000029"
LW R1, 164(R0) -- Read Hit. R1 -> X"0000002A"
LW R1, 168(R0) -- Read Hit. R1 -> X"0000002B"
LW R1, 172(R0) -- Read Hit. R1 -> X"0000002C"

-- Set 3
LW R1, 176(R0) -- Read Miss Clean (Reemplazo). R1 -> X"0000002D"
LW R1, 180(R0) -- Read Hit. R1 -> X"0000002E"
```

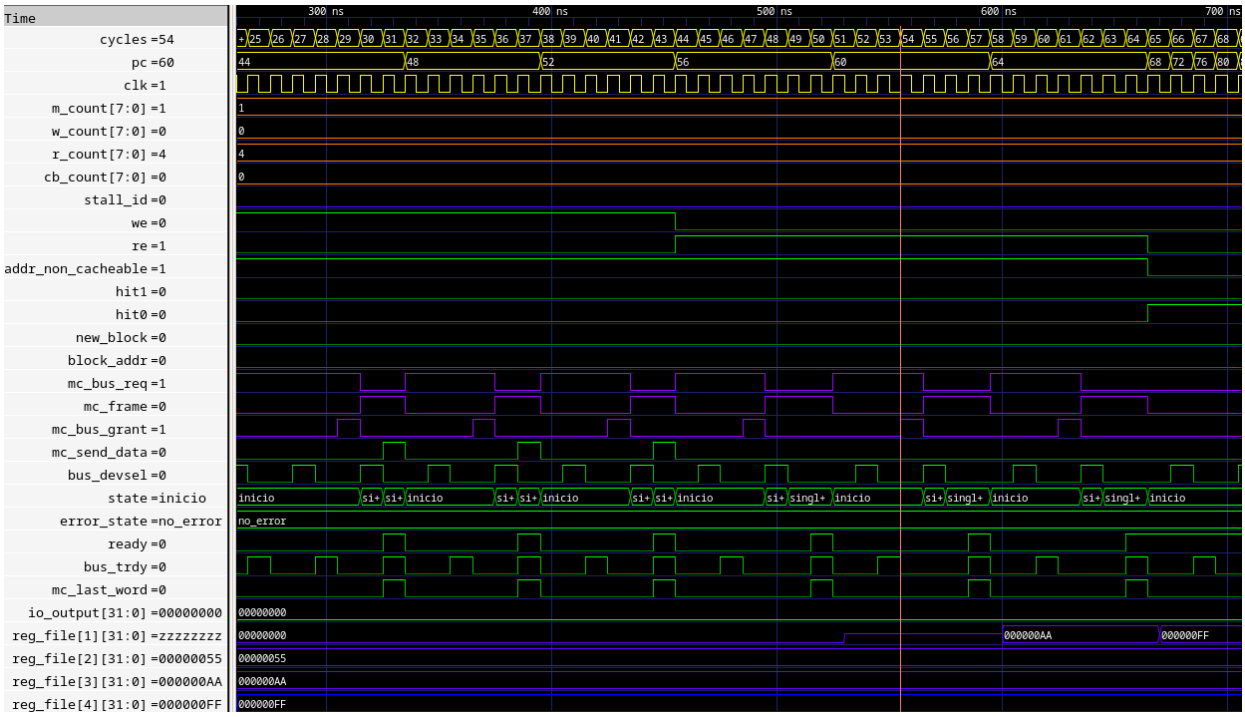

Time
cycles
pc
clk
m_count[7:0]
w_count[7:0]
r_count[7:0]
cb_count[7:0]
addr_non_cacheable
hit1
hit0
new_block
block_addr
mc_bus_req
mc_frame
mc_bus_grant
mc_send_data
bus_devsel
state
error_state
ready
bus_trdy
mc_last_word
re
we
io_output[31:0]
reg_file[1][31:0]
reg_file[2][31:0]
reg_file[3][31:0]
reg_file[4][31:0]
inicio
block_transfer_data
no_error


Time
cycles
pc
clk
m_count[7:0]
w_count[7:0]
r_count[7:0]
cb_count[7:0]
addr_non_cacheable
hit1
hit0
new_block
block_addr
mc_bus_req
mc_frame
mc_bus_grant
mc_send_data
bus_devsel
state
error_state
ready
bus_trdy
mc_last_word
re
we
io_output[31:0]
reg_file[1][31:0]
reg_file[2][31:0]
reg_file[3][31:0]
reg_file[4][31:0]
inicio
block_transfer_data
no_error


Signals
Waves
Time
1900 ns
2 us
2100 ns
2200 ns
2300
cycles =1
pc =4
clk =0
m_count[7:0] =0
w_count[7:0] =0
r_count[7:0] =0
cb_count[7:0] =0
addr_non_cacheable =0
hit1 =0
hit0 =0
new_block =0
block_addr =0
mc_bus_req =0
mc_frame =0
mc_bus_grant =0
mc_send_data =0
bus_devsel =1
state =i
error_state =n
ready =1
bus_trdy =0
mc_last_word =0
re =0
we =0
io_output[31:0] =0
reg_file[1][31:0] =0
reg_file[2][31:0] =0
reg_file[3][31:0] =0
reg_file[4][31:0] =0
inicio
block_transfer_data
no_error
00000026
00000000

Scratch (700ns)

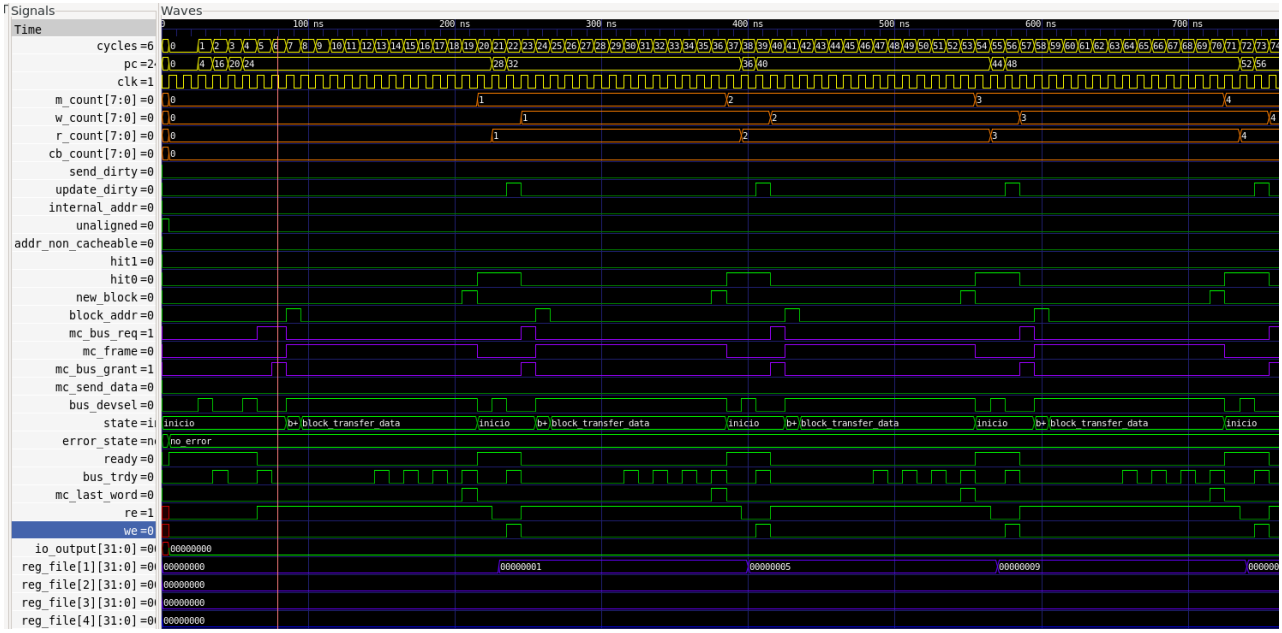
```
LW R5, 0(R0)      -- R5 = X"10000000" (Read Miss Clean)
LW R2, 4(R0)      -- R2 = X"00000055" (Read Hit)
LW R3, 8(R0)      -- R3 = X"000000AA" (Read Hit)
LW R4, 12(R0)     -- R4 = X"000000FF" (Read Hit)


SW R2, 0(R5)      -- Escribe X"55" en la palabra 0 de la Scratch (@ X"10000000")
Este dato sera modificado si o si porque IO_Master escribe ZZZs en la primera
direccion de la Scratch
SW R3, 4(R5)      -- Escribe X"AA" en la palabra 1 de la Scratch (@ X"10000004")
SW R4, 8(R5)      -- Escribe X"FF" en la palabra 2 de la Scratch (@ X"10000008")


LW R1, 0(R5)      -- Lee de Scratch. R1 -> Seguramente ZZZs
LW R1, 4(R5)      -- Lee de Scratch. R1 -> X"000000AA"
LW R1, 8(R5)      -- Lee de Scratch. R1 -> X"000000FF"
```



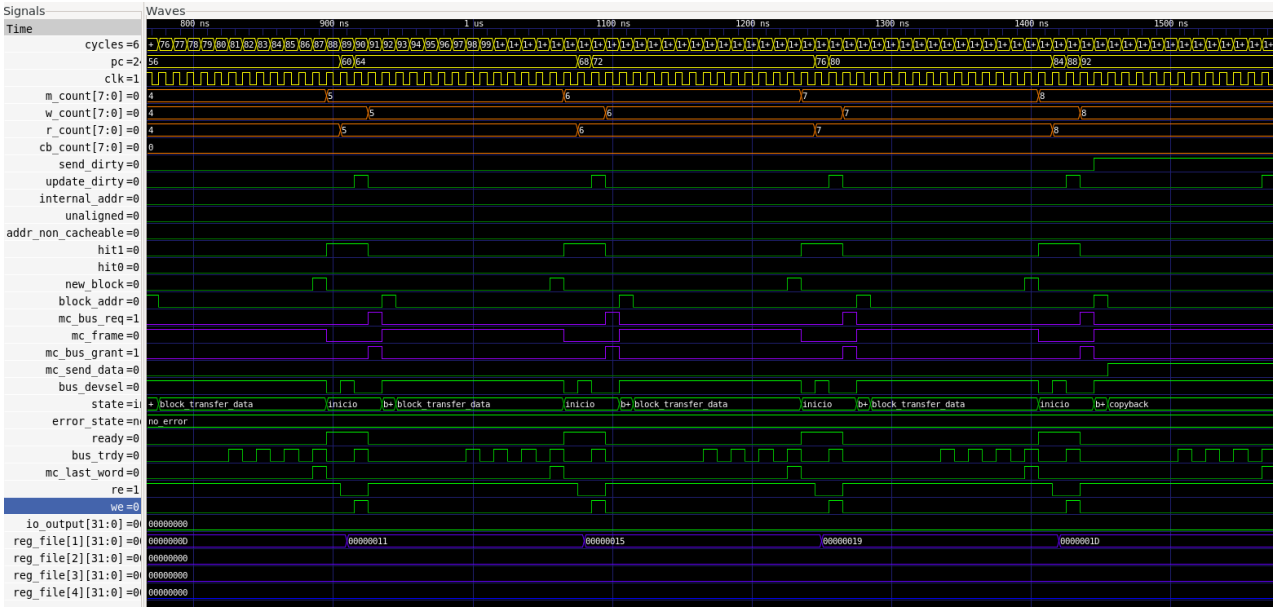
ReadMissDirty(4200ns)

```
-- Llenamos y ensuciamos la Vía 0 (Conjuntos 0 al 3)
LW R1, 0(R0) -- Read Miss Clean. Carga Dir 0 en Conjunto 0, Vía 0.
SW R1, 4(R0) -- Write Hit. Marca Conjunto 0, Vía 0 Dirty.
LW R1, 16(R0) -- Read Miss Clean. Carga Dir 16 en Conjunto 1, Vía 0.
SW R1, 20(R0) -- Write Hit. Marca Conjunto 1, Vía 0 Dirty.
LW R1, 32(R0) -- Read Miss Clean. Carga Dir 32 en Conjunto 2, Vía 0.
SW R1, 36(R0) -- Write Hit. Marca Conjunto 2, Vía 0 Dirty.
LW R1, 48(R0) -- Read Miss Clean. Carga Dir 48 en Conjunto 3, Vía 0.
SW R1, 52(R0) -- Write Hit. Marca Conjunto 3, Vía 0 Dirty.

-- Llenamos y ensuciamos la Vía 1 (Conjuntos 0 al 3)
LW R1, 64(R0) -- Read Miss Clean. Carga Dir 64 en Conjunto 0, Vía 1.
SW R1, 68(R0) -- Write Hit. Marca Conjunto 0, Vía 1 Dirty.
LW R1, 80(R0) -- Read Miss Clean. Carga Dir 80 en Conjunto 1, Vía 1.
SW R1, 84(R0) -- Write Hit. Marca Conjunto 1, Vía 1 Dirty.
LW R1, 96(R0) -- Read Miss Clean. Carga Dir 96 en Conjunto 2, Vía 1.
SW R1, 100(R0) -- Write Hit. Marca Conjunto 2, Vía 1 Dirty.
LW R1, 112(R0) -- Read Miss Clean. Carga Dir 112 en Conjunto 3, Vía 1.
SW R1, 116(R0) -- Write Hit. Marca Conjunto 3, Vía 1 Dirty.

LW R2, 128(R0) -- Read Miss Dirty (Set 0). Expulsa Vía 0 (Dir 0). CopyBack
LW R2, 144(R0) -- Read Miss Dirty (Set 1). Expulsa Vía 0 (Dir 16). CopyBack
LW R2, 160(R0) -- Read Miss Dirty (Set 2). Expulsa Vía 0 (Dir 32). CopyBack
LW R2, 176(R0) -- Read Miss Dirty (Set 3). Expulsa Vía 0 (Dir 48). CopyBack

LW R3, 192(R0) -- Read Miss Dirty (Set 0). Expulsa Vía 1 (Dir 64). CopyBack
LW R3, 208(R0) -- Read Miss Dirty (Set 1). Expulsa Vía 1 (Dir 80). CopyBack
LW R3, 224(R0) -- Read Miss Dirty (Set 2). Expulsa Vía 1 (Dir 96). CopyBack
LW R3, 240(R0) -- Read Miss Dirty (Set 3). Expulsa Vía 1 (Dir 112). CopyBack
```

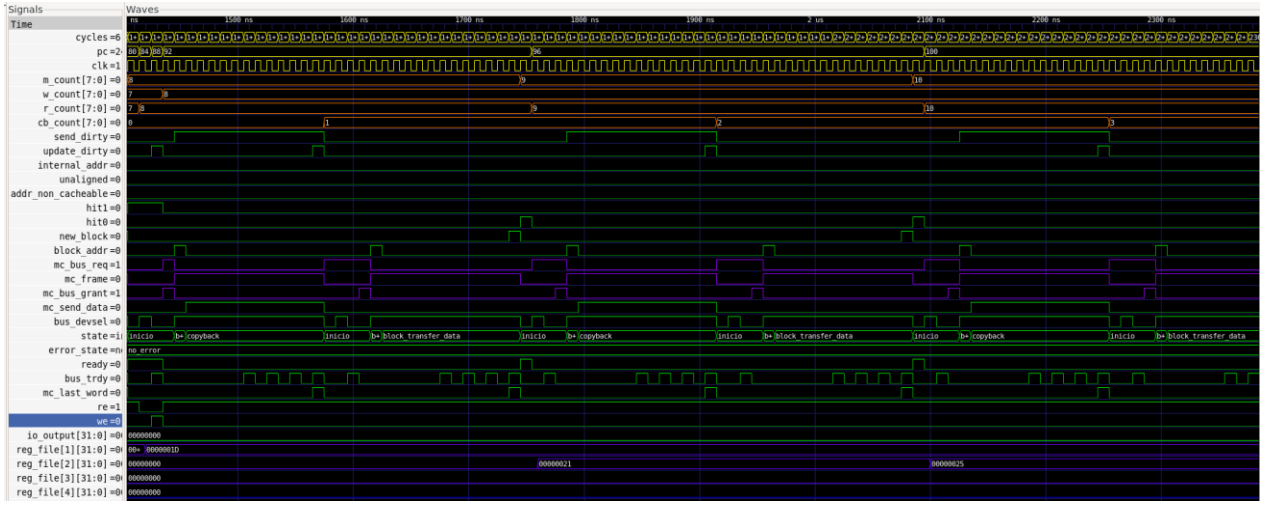
Llenamos vía 0



Llenamos vía 1

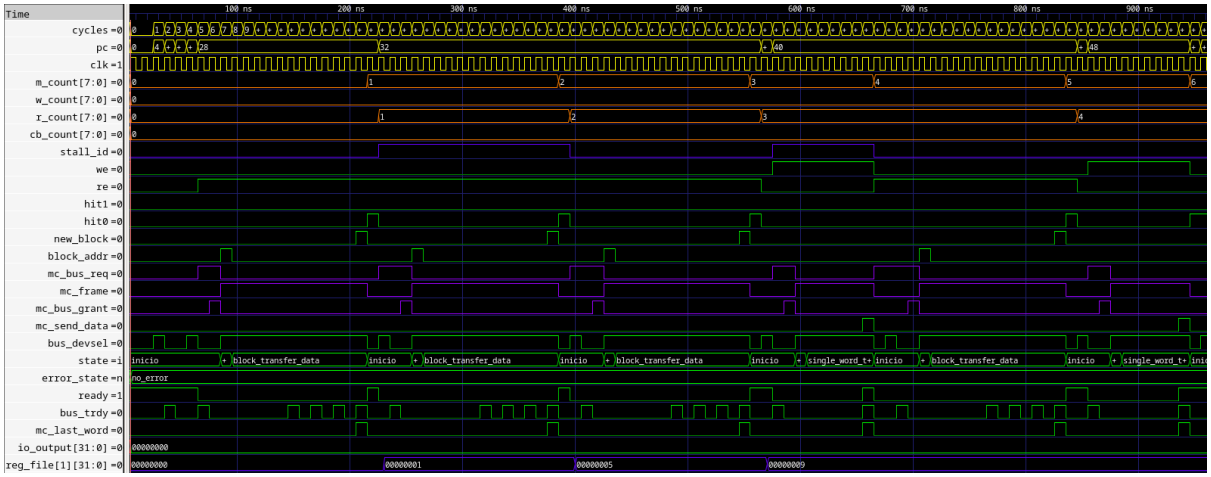


Reemplazos de la vía 0 sucia



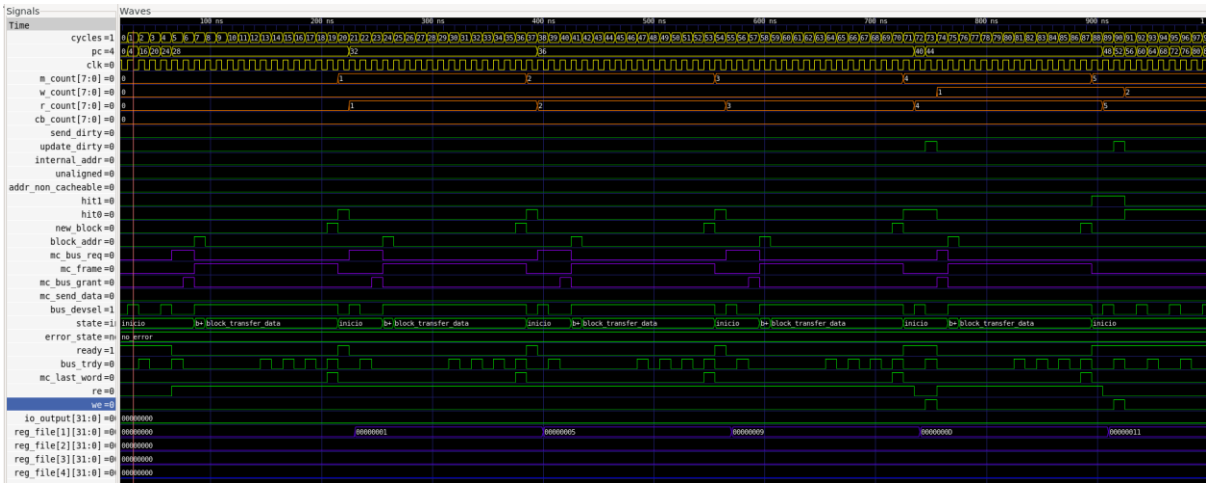
WriteMiss (1000ns)

```
-- Llenamos y ensuciamos la Vía 0 (Conjuntos 0 al 3)
LW R1, 0(R0) -- Fallo. Carga Dir 0 en Conjunto 0, Vía 0.
LW R1, 16(R0) -- Fallo. Carga Dir 16 en Conjunto 1, Vía 0.
LW R1, 32(R0) -- Fallo. Carga Dir 32 en Conjunto 2, Vía 0.
SW R1, 48(R0) -- Fallo. Escribe en MP
LW R1, 48(R0) -- Fallo. Carga Dir 48 en Conjunto 3, Vía 0.
SW R1, 64(R0) -- Fallo. Escribe en MP
```



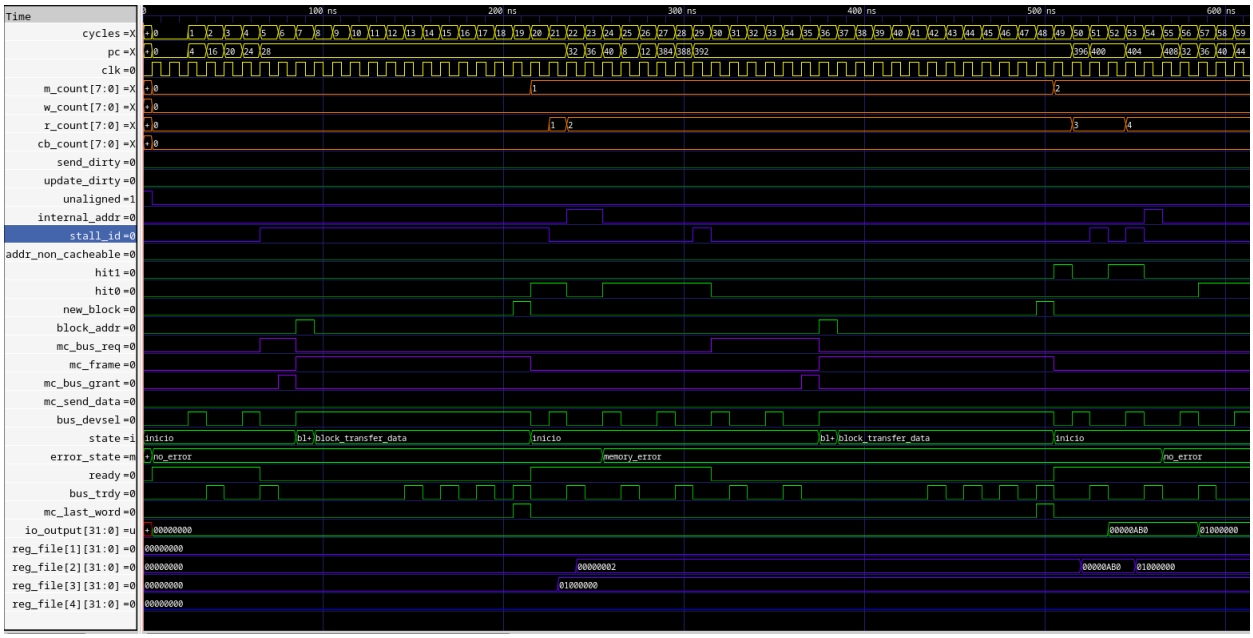
WriteHit(970ns)

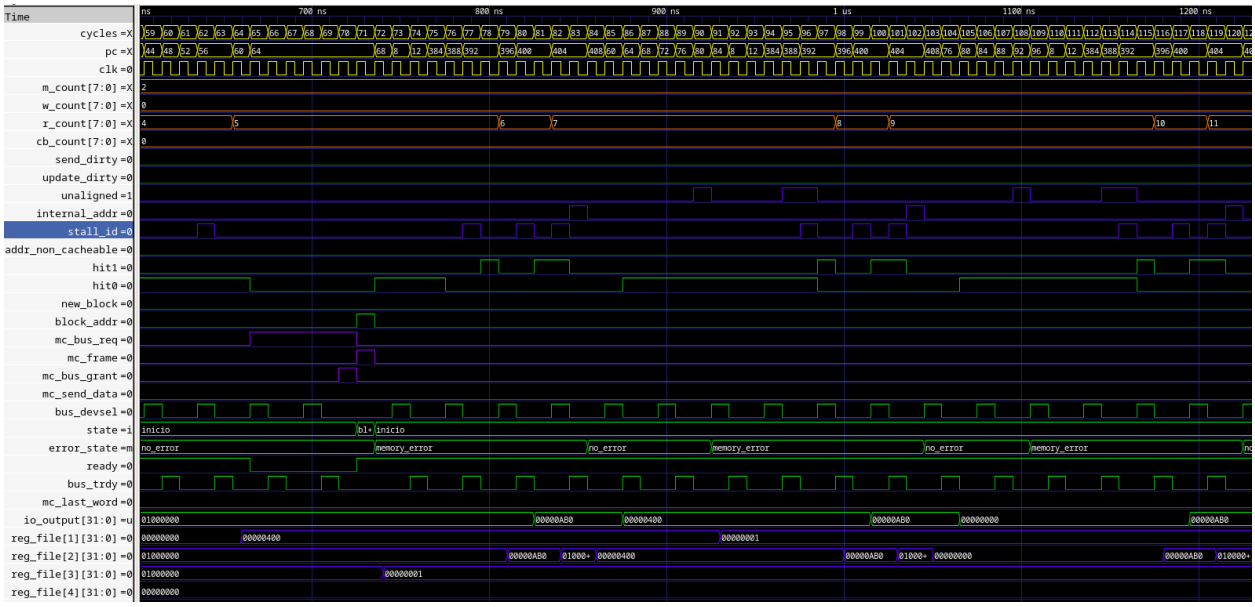
```
-- Llenamos y ensuciamos la Vía 0 (Conjuntos 0 al 3)
LW R1, 0(R0) -- Fallo. Carga Dir 0 en Conjunto 0, Vía 0.
LW R1, 16(R0) -- Fallo. Carga Dir 16 en Conjunto 1, Vía 0.
LW R1, 32(R0) -- Fallo. Carga Dir 32 en Conjunto 2, Vía 0.
LW R1, 48(R0) -- Fallo. Carga Dir 48 en Conjunto 3, Vía 0.
SW R1, 0(R0) -- Acierto.

LW R1, 64(R0) -- Fallo. Carga Dir 64 en Conjunto 0, Vía 1.
SW R1, 64(R0) -- Acierto.
```



Errores (1300 ns)

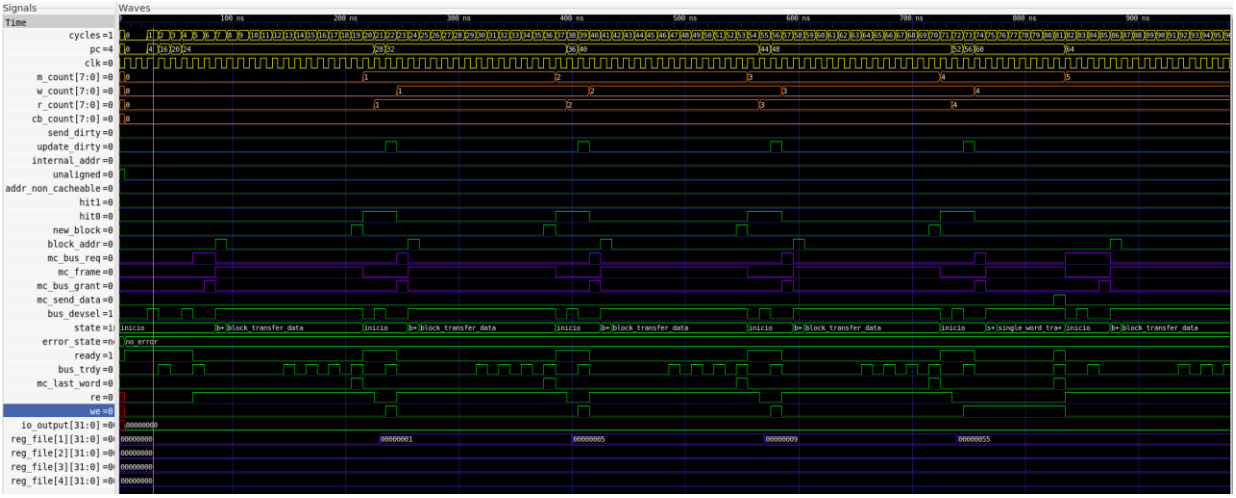
```
LW R3, 4(R0)     -- R3 = X"01000000" (Read Miss Clean)
LW R2, 8(R0)     -- R2 = X"00000002" (Read Hit)

SW R2, 0(R3)     -- Intento de escritura en registro interno. Activa Mem_ERROR

LW R1, 12(R0)    -- R1 = X"00000400" (Dirección inválida, fuera de rango)

LW R3, 0(R1)     -- Intento de lectura. Se activa Mem_ERROR y ocurre un dataAbort

LW R1, 1(R0)     -- Unaligned
SW R1, 2(R0)     -- Unaligned
```

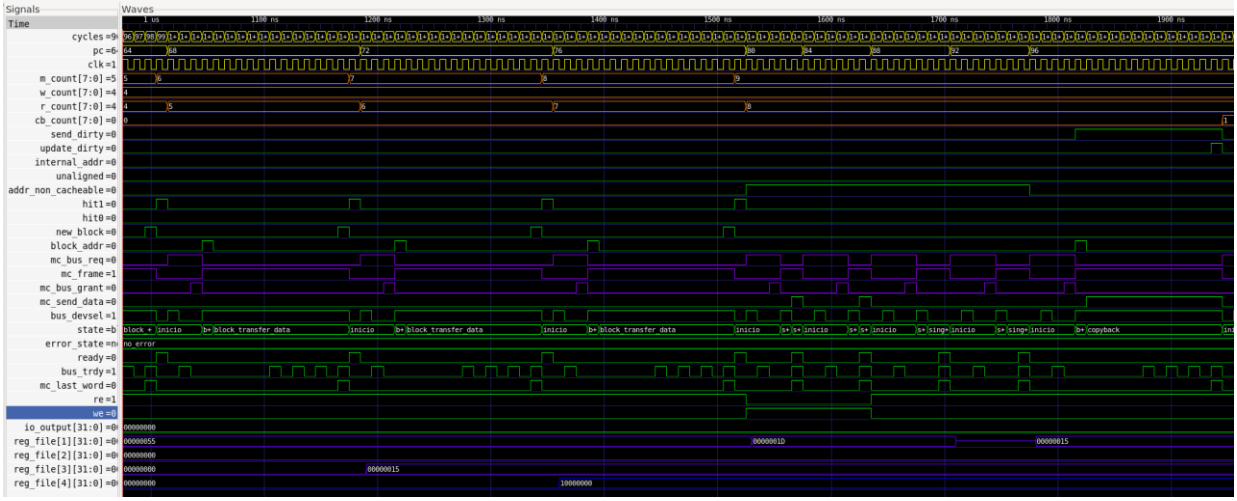


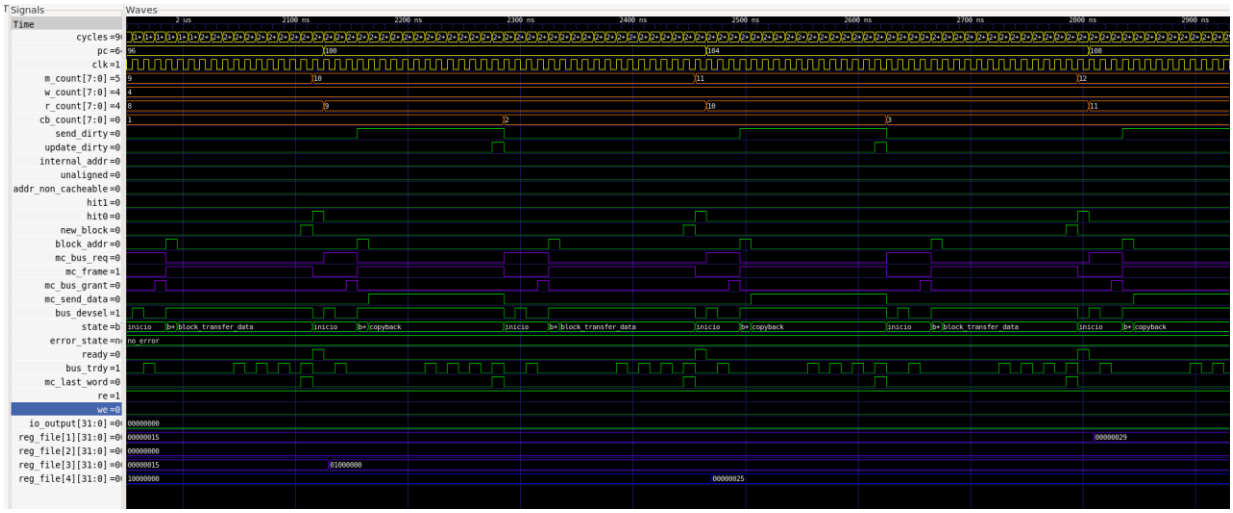


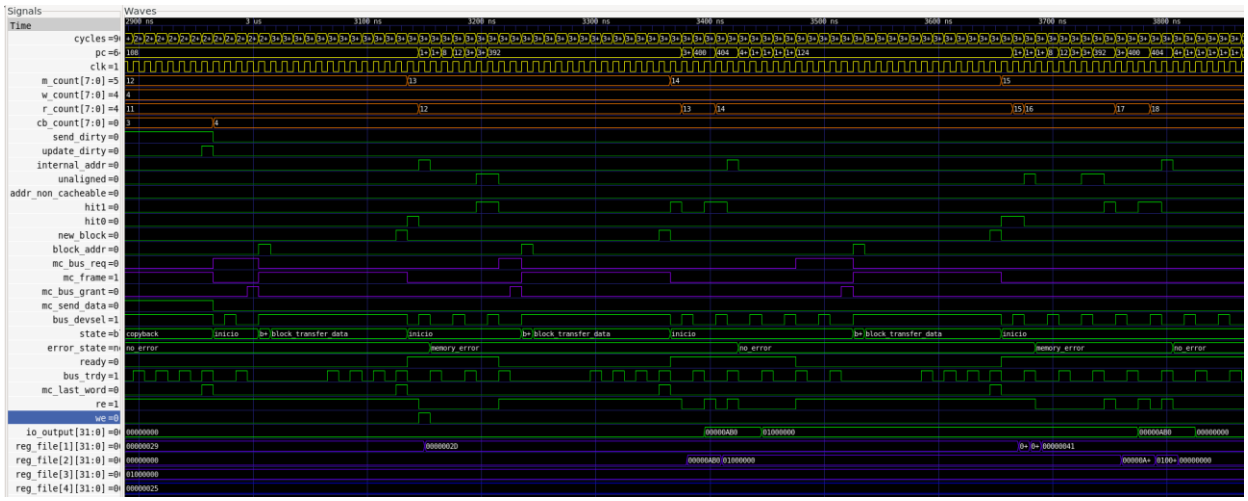
Completo(3900ns)

```
-- Llenamos y ensuciamos la Vía 0 (Conjuntos 0 al 3)
LW R1, 0(R0) -- Read Miss Clean. Carga Dir 0 en Conjunto 0, Vía 0.
SW R1, 4(R0) -- Write Hit. Marca Conjunto 0, Vía 0 Dirty.

LW R1, 16(R0) -- Read Miss Clean. Carga Dir 16 en Conjunto 1, Vía 0.
SW R1, 20(R0) -- Write Hit. Marca Conjunto 1, Vía 0 Dirty.

LW R1, 32(R0) -- Read Miss Clean. Carga Dir 32 en Conjunto 2, Vía 0.
SW R1, 36(R0) -- Write Hit. Marca Conjunto 2, Vía 0 Dirty.

LW R1, 48(R0) -- Read Miss Clean. Carga Dir 48 en Conjunto 3, Vía 0.
SW R1, 52(R0) -- Write Hit. Marca Conjunto 3, Vía 0 Dirty.

SW R1, 64(R0) -- Write Miss. Escribe en MP.

LW R1, 64(R0) -- Read Miss Clean. Carga Dir 64 en Conjunto 0, Vía 1.
LW R3, 80(R0) -- Read Miss Clean. Carga Dir 80 en Conjunto 1, Vía 1.
(X"00000055")
LW R4, 96(R0) -- Read Miss Clean. Carga Dir 96 en Conjunto 2, Vía 1. (@
X"10000000")
LW R1, 112(R0) -- Read Miss Clean. Carga Dir 112 en Conjunto 3, Vía 1.

SW R3, 0(R4) -- Escribe X"55" en la palabra 0 de la Scratch (@ X"10000000") Este
dato sera modificado si o si porque IO_Master escribe ZZZs en la primera
direccion de la Scratch
SW R3, 4(R4) -- Escribe X"55" en la palabra 1 de la Scratch (@ X"10000004")

LW R1, 0(R4) -- Lee de Scratch. R1 -> Seguramente leerá ZZZs
LW R1, 4(R4) -- Lee de Scratch. R1 -> X"00000055"

LW R3, 128(R0) -- Read Miss Dirty (Set 0). Expulsa Vía 0 (Dir 0). Copy-Back R3 =
X"01000000"
LW R4, 144(R0) -- Read Miss Dirty (Set 1). Expulsa Vía 0 (Dir 16). CopyBack
LW R1, 160(R0) -- Read Miss Dirty (Set 2). Expulsa Vía 0 (Dir 32). CopyBack
LW R1, 176(R0) -- Read Miss Dirty (Set 3). Expulsa Vía 0 (Dir 48). CopyBack

SW R4, 0(R3) -- Intento de escritura en registro interno. Activa Mem_ERROR

NOP
NOP

LW R1, 192(R0) -- Read Miss Clean (Set 2). Expulsa Vía 1.
LW R1, 196(R0) -- Read Hit (Set 2).
LW R1, 1(R0) -- Unaligned
```


Signals
Waves
Time
cycles
pc
clk
m_count[7:0]
w_count[7:0]
r_count[7:0]
cb_count[7:0]
send_dirty
update_dirty
internal_addr
unaligned
addr_non_cacheable
hit1
hit0
new_block
block_addr
mc_bus_req
mc_frame
mc_bus_grant
mc_send_data
bus_devsel
state
error_state
ready
bus_trdy
mc_last_word
re
we
io_output[31:0]
reg_file[1][31:0]
reg_file[2][31:0]
reg_file[3][31:0]
reg_file[4][31:0]
inicio
block_transfer_data
single_word_tra
no_error
00000000
00000001
00000005
00000009
00000055


Signals
Waves
Time
cycles
pc
clk
m_count[7:0]
w_count[7:0]
r_count[7:0]
cb_count[7:0]
send_dirty
update_dirty
internal_addr
unaligned
addr_non_cacheable
hit1
hit0
new_block
block_addr
mc_bus_req
mc_frame
mc_bus_grant
mc_send_data
bus_devsel
state
error_state
ready
bus_trdy
mc_last_word
re
we
io_output[31:0]
reg_file[1][31:0]
reg_file[2][31:0]
reg_file[3][31:0]
reg_file[4][31:0]
block +
inicio
block_transfer_data
copyback
no_error
00000000
00000055
0000001D
00000015
00000000
10000000

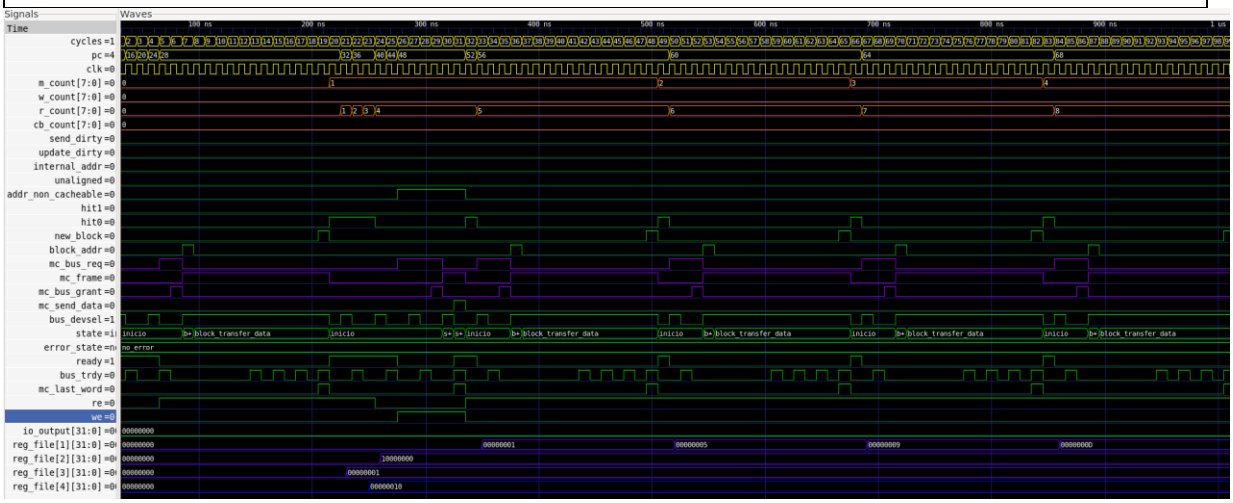


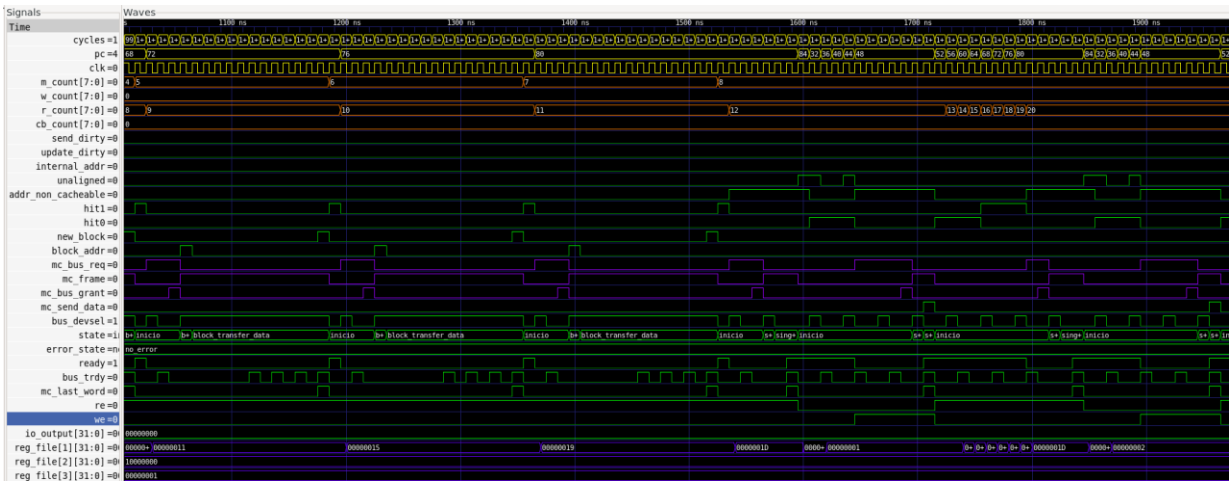


Bucle(5500ns)

```
LW R3, 0(R0)    -- Read Miss Clean. R1 -> X"00000001"
LW R1, 4(R0)     -- Read Hit (Carga Vía 0).   R3 -> X"00000000"
LW R4, 8(R0)     -- Read Hit. R4 -> X"00000016"
LW R2, 12(R0)    -- Read Hit. R2 -> X"10000000"

BUCLE:  BEQ R1, R4, FIN

SW R1, 4(R2)    -- Write Scratch (@X"10000004")

-- Vía 1
-- Set 0
LW R1, 0(R0)     -- Read Hit.   R1 -> X"00000001"
-- Set 1
LW R1, 16(R0)    -- Read Miss Clean (Carga Vía 0).   R1 -> X"00000005"
-- Set 2
LW R1, 32(R0)    -- Read Miss Clean (Carga Vía 0).   R1 -> X"00000009"
-- Set 3
LW R1, 48(R0)    -- Read Miss Clean (Carga Vía 0).   R1 -> X"0000000D"



-- Vía 1
-- Set 0
```

```
LW R1, 64(R0)    -- Read Miss Clean (Carga Vía 1).   R1 -> X"00000011"
-- Set 1
LW R1, 80(R0)    -- Read Miss Clean (Carga Vía 1).   R1 -> X"00000015"
-- Set 2
LW R1, 96(R0)    -- Read Miss Clean (Carga Vía 1).   R1 -> X"00000019"
-- Set 3
LW R1, 112(R0)   -- Read Miss Clean (Carga Vía 1).   R1 -> X"0000001D"

LW R1, 4(R2)    -- Read Scratch (@X"10000004") R1 -> Contador

ADD R1, R1, R3
BEQ R0 R0 BUCLE
FIN: BEQ R0 R0 -1
```





Prueba de integración

Para realizar esta prueba hemos escogido 2 códigos, nuestro código “Completo”, en el que observamos que tiene una pérdida respecto a la ejecución sin tener caché, debido a que en este código observamos una pequeña cantidad de hits y una notable cantidad de read miss dirty, que provocan una gran cantidad de ciclos. En esta segunda prueba aplicada sobre nuestro código” Bucle”, observamos una gran ganancia debido a que en el primer ciclo solamente se realizan misses, pero en las siguientes iteraciones son hits provocando una gran eficiencia.

Completo

Con Caché

$$\begin{aligned} Ciclos\ CC = {} & (14{,}5 + 1) \cdot 4 + 8{,}5 + 14{,}5 \cdot 4 + 3{,}5 \cdot 2 + 4{,}5 \cdot 2 + 29 \cdot 4 + 1 \\ & + (dataAbort: 14{,}5 + 2{,}5 \cdot 2) + 1 \cdot 2 + 14{,}5 + 1 + 2{,}5 \\ & + (dataAbort: 2{,}5 \cdot 3) = 310\ ciclos \end{aligned}$$

Sin Caché

$$\begin{aligned} CiclosSC = {} & (8{,}5 + 8{,}5) * 4 + 8{,}5 + 8{,}5 * 4 + 8{,}5 * 2 + 8{,}5 * 2 + 8{,}5 * 4 + 1 \\ & + (dataAbort: 8{,}5 \cdot 3) + 1 \cdot 2 + 8{,}5 \cdot 2 + 1 + (dataAbort: 8{,}5 \cdot 3) \\ & = 250{,}5\ ciclos \end{aligned}$$

SlowDown

$$SlowDown = \frac{250{,}5}{310} = 0{,}81$$

Bucle

Con Caché

$$\begin{aligned} CiclosCC = {} & 14{,}5 + 1 \cdot 3 + 1 + 3{,}5 + 1 + 14{,}5 \cdot 7 + 4{,}5 + 1 + 1 \\ & + (1 + 3{,}5 + 1 \cdot 8 + 4{,}5 + 1 + 2) \cdot 15 + 2 == 433\ ciclos \end{aligned}$$

Sin Caché

$$CiclosSC = 8{,}5 \cdot 4 + (1 + 8{,}5 + 8{,}5 \cdot 8 + 8{,}5 + 1 + 2) \cdot 16 + 2 = 1460\ ciclos$$

SpeedUp

$$SpeedUp = \frac{1460}{433} = 3{,}37$$

Se puede observar que el uso de la memoria caché aumenta el Speedup cuanto mayor es la localidad temporal y espacial.

Registro de tiempo (horas)

TAREA	MIJAYL	HÉCTOR
Diagrama de estados de la unidad de control + Programación de la UC	5:30	5:30
Desglose de direcciones	0:10	0:10
Análisis de la latencia de transferencia del bus	1:00	0:10
Ciclos promedio por acceso a memoria	1:30	0:00
Pruebas unitarias	3:10	10:30
Prueba de integración	3:15	1:30
Memoria	7:05	4:15
Total	21:30	21:55

Agradecimientos

Queremos agradecer a nuestro compañero Javier Martínez Virto nuevamente por el programa de codificación de instrucciones y a nuestro compañero Razvan Ghita Calangiu por intercambiar dudas sobre la depuración.

Autoevaluación individual

Mijayl Mandzyak Melnyk

• ¿Consideras que se han alcanzado los objetivos del curso?

Si, al final acabo es una consecuencia de todo el trabajo diario realizado a lo largo del cuatrimestre provocado por las entregas de problemas, prácticas y a ambos proyectos.

• ¿Qué calificación se daría a sí mismo/a?

Un 9.

Héctor Manzano Miranda

• ¿Consideras que se han alcanzado los objetivos del curso?

Si, considero que gracias a ambos proyectos, los problemas semanales y el material impartido en clase hemos alcanzado los objetivos de aprendizaje del curso.

• ¿Qué calificación se daría a sí mismo/a?

Un 9.